

String Class



String

La classe String no és un tipus de dada primitiu com ho és `int` o `double`, sinó que les dades de tipus String en Java són objectes.

Java proveeix la classe `String` per manipular i crear cadenes de caràcters.

Creació de Strings

Es defineixen entre cometes doubles.

Exemple de creació de cadenes de caràcters.

```
String st1 = "Joan";  
String st2 = "This is a sentence with some words."  
String st3 = "This is a sentence with " +  
             "more than one line. " +  
             "THE END."
```

Output:

```
st1: Joan  
st2: This is a sentence with some words.  
st3: This is a sentence with more than one line. THE END.
```

Els objectes de la classe **String** es poden crear, llegir i modificar, però cal tenir en compte que quan es modifica un String, Java elimina l'objecte i crea un altre objecte de tipus String amb el nou contingut.

Text blocs:

A partir de la versió Java 15, els strings es poden crear amb 3 cometes dobles `"""` a l'inici i tancament del bloc de text per preservar l'estil, tenir un text més llegible i evitar `"\n"`:

```
String text = """
    This is the first line.
        This is the second line.
    This is the last line.      THE END!
    """;

System.out.println(text);
```

Output:

```
This is the first line.
    This is the second line.
This is the last line.      THE END!
```

String mètodes

String no és una classe estàtica, com ho era la classe **Math**.

Quan utilitzem un mètode de la classe **Math**, per exemple `Math.round()`, la mateixa paraula **Math** s'ha d'utilitzar davant del mètode.

En canvi, la classe **String** no és estàtica i, per tant, haurem d'instanciar un objecte de la classe **String** i utilitzar aquest objecte per cridar qualsevol dels mètodes de la classe **String**.

length()

```
nom.length()
```

Retorna la longitud de la cadena.

Exemple: contar caràcters d'una cadena

```
String nom = "Aquesta cadena té una longitud de: ";
int llargada = nom.length();
System.out.println(nom + llargada);
```

Output:

```
Aquesta cadena té una longitud de: 34
```

charAt()

```
nom.charAt(n)
```

Retorna el caràcter que hi ha a la n posició de la cadena nom.

Exemple: trobar el que hi ha a la 3a posició

```
String nom = "Posicio 2 tenim la s";  
char u = nom.charAt(2);  
System.out.println(u);
```

Output:

```
s
```

Exemple: trobar el que hi ha a l'última lletra

```
String nom = "Aquesta cadena acaba amb x";  
char ultima lletra = nom.charAt(nom.length() - 1);  
System.out.println(ultima lletra);
```

Output:

```
x
```

substring()

```
nom.substring(x,y)
```

Retorna una subcadena dins d'una cadena, des de X i Y caràcters. * x la posició que volem * y a partir de la posició que ja no volem

Exemple: trobar el que hi ha a la 3a posició

```
String s1 = "Això és un text molt llarg.";  
String s2 = s1.substring(11, 20);  
System.out.println(s2);
```

Output:

```
text molt
```

equals()

```
s1.equals(s2)
```

Retorna true si les cadenes (s1 i s2) són iguals, distingeix entre minúscules i majúscules.

Exemple:

```
String s1 = "Això és un text molt llarg.";
String s2 = "Això és un text molt llarg.";
boolean si_no = s1.equals(s2);
System.out.println(si_no);
```

Output:

```
true
```

equalsIgnoreCase()

```
s1.equalsIgnoreCase(s2)
```

Retorna true si les cadenes (s1 i s2) són iguals sense diferenciar entre minúscules i majúscules.

Exemple:

```
String s1 = "Això és un text molt llarg.";
String s2 = "això és un text molt llarg.";
boolean si_no = s1.equalsIgnoreCase(s2);
System.out.println(si_no);
```

Output:

```
true
```

concat()

```
s1.concat(s2)
```

Concatenar Strings.

Exemple:

```
String s1 = "First sentence.";
String s2 = "Second sentence.";
String contrasenya = s1.concat(s2);
System.out.println(contrasenya);
```

Output:

```
First sentence.Second sentence.
```

contains()

```
s1.contains(s2)
```

Retorna `true` si s1 conté s2 .

Exemple:

```
String s1 = "A sentence with a sentence inside, to check if 12345 is inside  
the string.";  
String s2 = "12345";  
boolean b = s1.contains(s2);
```

Output:

```
true
```

replace()

```
replace('a', 'b')
```

Reemplaçar el caràcters 'a' pel 'b'.

Exemple:

```
String contrasenya= "America5682";  
contrasenya = contrasenya.replace('a', 'e').replace('A', 'E').replace('5',  
'0');  
System.out.println(contrasenya);
```

Output:

```
Emalice0682
```

repeat()

```
repeat(int)
```

Repeteix el string les vegades que s'indica per paràmetre.

Exemple:

```
String text = "*";  
String result = text.repeat(10);  
System.out.println(result);
```

Output:

```
*****
```

toCharArray()

```
cadena.toCharArray()
```

Convertir un String a un array de Chars.

Exemple:

```
String str= "Això és un text molt llarg.";  
char[] chars = str.toCharArray();
```

La inversa, crear un String a partir d'array de caràcters:

```
char[] hellowArray = {'h','e','l','l','o'};  
String str1 = new String(hellowArray);
```

toLowerCase()

```
cadena.toLowerCase()
```

Convertir un String a minúscules.

Exemple:

```
String str= "Això és un text molt llarg.";  
String s = "THIS IS UPPERCASE";  
String s1 = s.toLowerCase();  
System.out.println(s1);
```

Output:

```
this is uppercase
```

toUpperCase()

```
cadena.toUpperCase()
```

Convertir un String a majúscules.

Exemple:

```
String s = "this is lowercase";  
String su = s.toUpperCase();  
System.out.println(su);
```

Output:

```
THIS IS UPPERCASE
```

compareTo()

```
s1.compareTo(s2)
```

Compara dos strings, si són iguals retorna 0, si s1 és més gran que s2 retorna >0 i si no <0.

Exemple:

```
String s1 = "this is a string";  
String s2 = "this is a string with more chars";  
  
if (s1.compareTo(s2) > 0) {  
    System.out.print("La primera cadena va després de la segona.");  
} else if (s1.compareTo(s2) < 0) {  
    System.out.print("La primera cadena va abans de la segona.");  
} else {  
    System.out.print("Les cadenes són iguals.");  
}
```

Output:

```
La primera cadena va abans de la segona.
```

compareToIgnoreCase()

```
s1.compareToIgnoreCase(s2)
```

Compara dos strings ignorant si són majúscules o minúscules, si són iguals retorna 0, si s1 és més gran que s2 retorna >0 i si no <0.

Exemple:

```
String s1 = "THIS IS A STRING";
String s2 = "this is a string";

if (s1.compareToIgnoreCase (s2) > 0) {
    System.out.print("La primera cadena va després de la segona.");
} else if (s1.compareToIgnoreCase(s2) < 0) {
    System.out.print("La primera cadena va abans de la segona.");
} else{
    System.out.print("Les cadenes són iguals.");
}
```

Output:

```
Les cadenes són iguals.
```

trim()

```
s.trim()
```

Esborra els espais en blanc al davant i al darrera del string s.

Exemple:

```
String s1 = " abcde ";
String s2 = s1.trim();
System.out.println(s2);
```

Output:

```
abcde
```

indexOf()

```
s1.indexOf(s2)
```

Retorna la posició de s1 on es troba la s2 la 1a vegada, en cas de no trobar-se retorna -1.

Exemple:

```
String s1 = "this is a string";
String s2 = "is";
int index = s1.indexOf(s2);
System.out.println(index);
```

Output:

```
2
```

També es pot realitzar amb un caràcter com a paràmetre:

```
s.indexOf(char)
```

Retorna la posició de la s on es troba el char la 1a vegada, en cas de no trobar-se retorna -1.

Exemple:

```
String s1 = "this is a string";  
char char1 = 'i';  
int index = s.indexOf(char1);  
System.out.println(index);
```

Output:

```
2
```

String.valueOf()

```
String.valueOf(n)
```

Converteix n a `String`, n pot ser `char`, `boolean`, `int`, `long`, `float` o `double`.

Nota: aquest mètode a diferència dels altres és un mètode estàtic, per tant, s'ha d'utilitzar el nom de la classe (`String`) al davant del mètode.

Exemple:

```
int n1 = 100;  
String s1 = String.valueOf(n1);  
System.out.println(s1);  
  
float n2 = 100f;  
String s2 = String.valueOf(n2);  
System.out.println(s2);  
  
double n3 = 100.24;  
String s3 = String.valueOf(n3);  
System.out.println(s3);
```

Output:

```
100  
100  
100.24
```

License and Copyright

Copyright (C) Joan Sèculi (joan.seculi@gmail.com).

Permission is granted to copy, distribute, and/or modify this document under the terms of the **GNU Free Documentation License**, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts.

A copy of the license is available at: <http://www.gnu.org/licenses/fdl.txt>